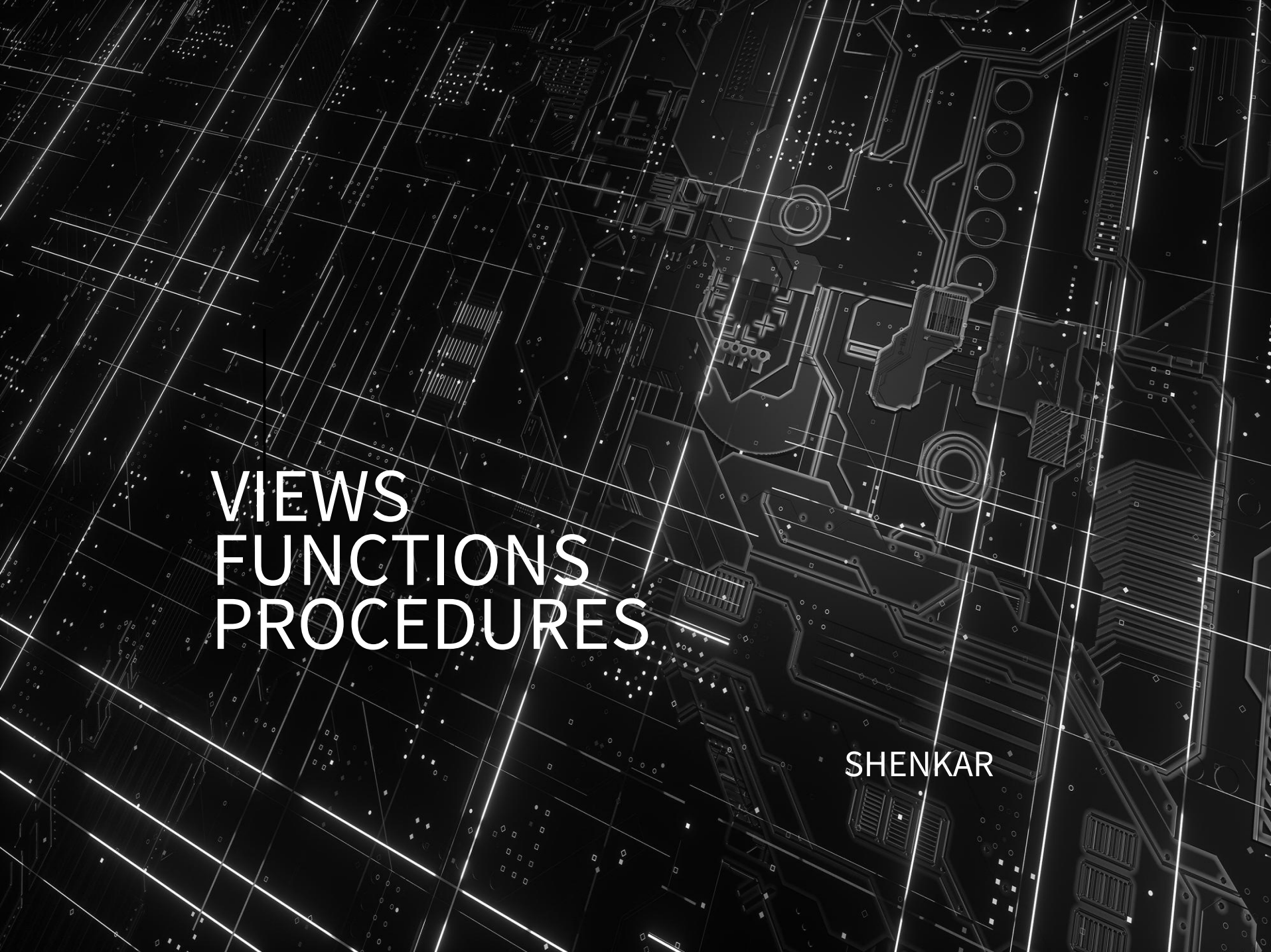
VIEWS
FUNCTIONS
PROCEDURES
SHENKAR

SQL VIEWS

- VIEW הוא שאילתה שניתן להשתמש בה כמו בטבלה רגילה.
  נשתמש ב VIEWS:
  - שאילתות מורכבת
  - משיקולי אבטחה (security)

EXAMPLE SQL VIEW

- A SQL view to show actors and their films:

```
CREATE VIEW actor_film_vw AS
SELECT
actor.first_name, actor.last_name, film.title film_id
FROM
actor
INNER JOIN
        film_actor fa
ON actor.actor_id = fa.actor_id
INNER JOIN
film
        ON film.film_id = fa.film_id;
```

  - Views can take arguments and return values

EXAMPLE SQL VIEW

- Can use our view for simple select:

```
SELECT * from actor_film_vw;
```

- Can include in join queries:

```
SELECT    ac.*, c.name category
FROM    actor_film_vw ac
INNER JOIN    film_category fa ON ac.film_id = fa.film_id
INNER JOIN    category c ON fa.category_id = c.category_id          ;
```

- Can include in **WHERE**clause:

```
SELECT    first_name, last_name, COUNT(*)
    num_of_film
    FROM    actor_film_vw
    WHERE
    first_name = 'KEVIN’
    GROUP BY first_name , last_name;
```

SQL FUNCTIONS

- שאילתות יכולות לבצע פעולות מורכבות ולהשתמש בפונקציות מובנות.
- הרבה פעמים נדרש לבצע פעולות מורכבות וחוזרות.
- מערכת ה DBMS – מאפשרת לבנות: User-Defined Functions (UDFs)

EXAMPLE SQL FUNCTION

- פונקציה שסופרת את כמות הסרטים עבור שחקן מסויים:

```
CREATE FUNCTION
        actor_count_fn(in_first_name VARCHAR(45),in_last_name VARCHAR(45))
RETURNS INTEGER
BEGIN
    DECLARE a_count INTEGER;
    SELECT COUNT(*) INTO a_count
    FROM actor inner join
    film_actor fa
        on
            actor.actor_id = fa.actor_id
    WHERE actor.first_name = in_first_name and actor.last_name = in_last_name ;
    RETURN a_count;
END
```

- Function can take arguments and return values
- Use SQL statements and other operations in body of function

EXAMPLE SQL FUNCTION – DIFFERENT USE CASE

- שימוש ישיר על שחקן מסויים:

```
SELECT actor_count_fn('NICK','WAHLBERG');
```

- שימוש כחלק משאילתה:

```
SELECT first_name, last_name ,
actor_count_fn(first_name, last_name) AS number_of_film
FROM actor;
```

- שימוש כחלק מתנאי ה WHERE:

```
SELECT first_name, last_name
  ,actor_count_fn(first_name,
  last_name) AS number_of_film
  FROM actor WHERE
  actor_count_fn(first_name,
  last_name) BETWEEN 20 AND 25;
```

ARGUMENTS AND RETURN-VALUES

- Functions can take any number of arguments
- Functions *must* return a value
  - Specify type of value in **RETURNS clause**

- From our example:

```
CREATE FUNCTION
actor_count_fn(in_first_name VARCHAR(45),in_last_name VARCHAR(45))   RETURNS INTEGER
```

  - The arguments named **in_first_name**, **in_last_name** type is **VARCHAR(45)**
  - Returns some **INTEGER**value

FUNCTION'S BODY

- Our **actor_count_fn** function's body:

```
BEGIN
    DECLARE a_count INTEGER;
    SELECT COUNT(*) INTO a_count
    FROM actor inner join
    film_actor fa
            on
                    actor.actor_id = fa.actor_id
    WHERE actor.first_name = in_first_name and actor.last_name =
    in_last_name ;
    RETURN a_count;
END
```

- A simple integer variable with initial value:

```
BEGIN
  DECLARE a_count INTEGER DEFAULT 0;
  ...
END
```

VARIABLES:

- Can use **SELECT**… **INTO**syntax
  - For assigning the result of a query into a variable

```
SELECT COUNT(*) INTO a_count  FROM actor
  INNER JOIN
  film_actor fa ON actor.actor_id =
  fa.actor_id
  WHERE    actor.first_name = in_first_name
  AND actor.last_name = in_last_name;
```

  - Query must produce a single row
- Can also use **SET**syntax
  - For assigning result of a math expression to a variable

```
SET result = n * (n + 1) / 2;
```

EXAMPLES:

- Simple function to compute sum of 1..N

```
CREATE FUNCTION sum_numbers(n INTEGER) RETURNS
INTEGER
BEGIN
  DECLARE result INTEGER DEFAULT 0;
  SET result = n * (n + 1) / 2;
  RETURN result;
END
```

- To simplify:

```
CREATE FUNCTION sum_numbers_simple (n INTEGER)
RETURNS INTEGER
BEGIN
  RETURN n * (n + 1) / 2;
END
```

SQL PROCEDURES (STORED PROCEDURES)

- Functions have specific limitations
  - Must return a value
  - All arguments are input-only
  - Typically cannot affect transaction status (i.e. commit, rollback, etc.)
  - Usually not allowed to modify relations, except in particular circumstances
- Procedures are more general constructs without these limitations
  - Also generally cannot be used in same places as functions

EXAMPLE PROCEDURE

- Write a procedure that returns both the number of films an actor has, and their total length
  - Results are passed back using out-parameters

```
CREATE PROCEDURE category_summary_proc(
     IN in_category_name VARCHAR(20))
     BEGIN
     SELECT      c.name category_name,
     count(*) num_of_movies,
     sum(film.length) total_length    FROM
     film            INNER JOIN
     film_category fc ON film.film_id =
     fc.film_id            INNER JOIN
     category c ON fc.category_id =
     c.category_id        WHERE
     c.name = in_category_name
     GROUP BY C.NAME;
     END
```

- Default parameter type is **IN**

CALLING A PROCEDURE

- Use the **CALL** statement to invoke a procedure

```
CALL category_summary_proc('action');
```

- To use this procedure with output parameters, must also have variables to receive the values
- MySQL SQL syntax:

```
category_summary_out_proc('action',@cat,@num,@length);

select @cat,@num,@length;


# @cat, @num, @length
'Action', '64', '7143'
```

EXERCISES

- Create stored procedure:
  input: actor name: first name and last name
  output (as select): actor name, number of his films, total length of all his films.

  In addition, every time the procedure will execute insert into new table – proc_log – the input and time.